Work Split — 4 Roles, Clear Boundaries

Generative-AI Embodied Robot Assistant · CSCI444/944 · due Sunday 1 November 2026, 11:30 pm

Principle

Each role owns **one layer** of the architecture (see `diagrams/1_architecture.drawio`). It talks to the other layers **only through agreed interfaces** (§2). Every role can be built and tested against mocks before integration. Each role is about **14–15 effort points** (1 pt ≈ ½ day). Ownership means you write it, test it, merge it and explain it in the report. It does **not** mean nobody else may read it: the spec (§15) lets the marker ask **any** member about **any** component.

1 · Interaction
speech, dialogue, Supervisor
≈ 14 pts

2 · Planning
LLM, prompts, replanning loop
≈ 14 pts

3 · Safety & Execution
validator, executive, state, skills glue
≈ 15 pts

4 · Robot Skills
navigation, perception, search
≈ 14 pts

1 · File ownership map

Every file in the starter package has exactly **one owner**. Only the owner merges changes to it, and others open a pull request or ask.

File	Owner	Notes
`speech_io.py`, `nlu_dialogue.py`	R1	ASR / TTS, `DialogueState`, intent routing
`controllers/setup_supervisor/` (new) + its node in the `.wbt`	R1	Temporary until staff confirm; spawn points live **inside** this folder
`llm_interface.py`, `prompts/`, `config/llm_config*.json`	R2	All model-specific imports stay here
`plan_schema.py`, `plan_validator.py`	R3	The safety gate. Nobody else edits it, and R2 in particular must not
`task_executive.py`, `world_state.py`, `semantic_memory.py`, `robot_skills.py`	R3	Executive, evidence store and the skill dispatcher
`project3_student.py` (main loop)	R3 + sections	R3 owns and merges. R1 adds the **speech-queue poll**, R2 adds the **replanning block**
`navigation.py`, `perception.py`	R4	All motor and camera access goes through these
`config/locations.json`, `config/navigation_graph.json`	R4	—
`config/objects.json`	R3	Must stay in sync with `plan_schema.py`
`tests/test_plan_validator.py`, `tests/test_executive.py`	R3	—
`tests/test_dialogue.py` · `tests/test_llm_parsing.py` · `tests/test_perception.py`	R1 · R2 · R4	Each role adds its own test file
`README.md`, `requirements.txt`	R1	Everyone sends their dependencies and run steps to R1

2 · Interface contracts (freeze on day 1–2)

These are the **only** points where roles touch each other. The formats below are a **proposal**: agree on them together, write them into `plan_schema.py` or the README, and don't change them without telling the group.

Interface	Producer → Consumer	Agreed format (proposal)
Plan dict	R2 → R3	Spec §7 contract v1.0 (`version`, `goal`, `steps`). Any extension needs R3's validator update and a test first
Validation result	R3 → R2	`(valid: bool, errors: list[str])`; the errors go back into the corrected-plan prompt
Skill status	R4 → R3	One of `"RUNNING"`, `"SUCCEEDED"`, `"FAILED"`, `"TIMEOUT"`, `"BLOCKED"`, `"CANCELLED"`; define the constants once in `plan_schema.py`
Navigator API	R4 → R3	`start(location) → status`, `update() → status`, `cancel()`, plus an evidence dict (position, reason, elapsed)
Search evidence	R4 → R3	`update_search() → (status, evidence)` with `evidence = {result: FOUND | NOT_FOUND | UNCERTAIN, confidence, views_done, frames, location}`

Interface	Producer → Consumer	Agreed format (proposal)
`WorldState` fields	R3 → R1, R2	The `as_dict()` keys. R1 builds status answers from them and R2 puts them in prompts
Failure trace	R3 → R2	Bounded list: last N steps, each `{step, status, evidence}`; this is the input to `replan()`
Speech input queue	R1 → main loop	`queue.Queue` of `{kind: goal | answer | status | stop, text}`
`speak(text)`	R1 → R3, main loop	Non-blocking; returns at once

3 · Roles

Role 1 · Interaction

Member: _____

Owns: `speech_io.py`, `nlu_dialogue.py`, setup Supervisor, README
Leads marks: Speech & NL (3) · explanation in Robustness (2)

Task (definition of done)	Pts
ASR (Vosk or Whisper) in a worker thread → input queue; never blocks `robot.step()`	3
Non-blocking TTS: `speak()` returns at once and speech plays in its own thread	2
`analyse_for_missing_information()`: rules for a missing colour, object or location ("find the ball" → "which colour?")	2
`consume_followup()`: merge the answer with `pending_goal` and clear the waiting state	1
Intent routing: new goal / clarification answer / status question / stop; busy-goal policy	2
Status answer ("What are you doing and why?") built only from `WorldState.as_dict()`	1
Speech-queue poll in the main loop → `submit_goal()` (your section of `project3_student.py`)	1
Temporary setup Supervisor based on Lab 2 `ball_randomizer` (replace it if staff supply one)	2

Provides to others
Input queue messages · `speak()` · clarified goal text · Supervisor for evaluation trials

Consumes from others
`WorldState` keys (R3) · cancel entry point `executive.cancel()` (R3)

Boundary — does NOT: call the LLM or validator directly · touch motors or devices · give the controller any access to the Supervisor's spawn point or log

Own tests: `tests/test_dialogue.py`: ambiguous, complete and follow-up goals; status and stop routing

Role 2 · Planning

Member: _____

Owns: `llm_interface.py`, `prompts/`, `config/llm_config*.json`
Leads marks: GenAI interpretation (3) · Plan generation (3)

Task (definition of done)	Pts
Backend client (llama-cpp Qwen, or an online model) loaded from config; timeout; credentials from env vars only	2
Prompt: fill the template placeholders and add few-shot examples (sequence, conditional, constrained order, unsafe)	3
Parsing: extract and repair JSON → `dict`; keep raw response, parsed plan and parse errors (log to `logs/`)	2
`replan()`: prompt with goal, bounded failure trace and remaining steps; ask for a **remainder** plan only	2
Mock backend and saved responses, clearly labelled as fixtures	1
Phrasing test set: 8 request types × 3–4 new phrasings; measure how often plans validate	2
Replanning block in the main loop: on FAILED → `replan()` → validate → restart, stop after `MAX_REPLANS`	2

Provides to others
Plan dicts (untrusted) · revised plans · raw-response logs · plan-validity numbers for the report

Consumes from others
Validator errors (R3) · failure trace and `WorldState` (R3) · clarified goal (R1)

Boundary — does NOT: decide that a plan is safe · edit `plan_validator.py` or `plan_schema.py` · import Webots · emit motor commands · let backend-specific code leak outside `llm_interface.py`

Own tests: `tests/test_llm_parsing.py`: valid JSON, Markdown-wrapped JSON, broken JSON and the replan prompt (mock backend)

Role 3 · Safety & Execution

Member: _____

Owns: `plan_schema.py`, `plan_validator.py`, `task_executive.py`, `world_state.py`, `semantic_memory.py`, `robot_skills.py`, main loop
Leads marks: Validation & grounding (3) · Monitoring & replanning (3)

Task (definition of done)	Pts
Validator: parameter types, `else` required, condition `value` types, unknown extra keys rejected	2
Validator: semantic checks (repeated navigation, contradictions, retry loops) and prohibited content (motor, code, shell, file, network)	3
Validator tests: valid, malformed, unsupported, unsafe, conditional and size-limit plans	2
`_expand_conditional()`: evaluate `{source, operator, value}` with fixed code (no `eval()`) and splice in the chosen branch	2
Executive statuses TIMEOUT, BLOCKED and CANCELLED; timeout per step; trace with evidence; `cancel()` path	2
`WorldState` extensions (`route_status`, structured search result); executive tests with fake skills	2
`robot_skills.py`: map R4's results to statuses, write evidence into `WorldState`, implement `observe` with `SemanticMemory`	2

Provides to others
Validated plans only · statuses and trace · `WorldState` · integration of the main loop

Consumes from others
Plan dicts (R2) · Navigator and search API with evidence (R4) · `speak()` (R1)

Boundary — does NOT: write navigation or perception algorithms · write prompts · trust any LLM field without checking it

Own tests: `tests/test_plan_validator.py`, `tests/test_executive.py`

Role 4 · Robot Skills

Member: _____

Owns: `navigation.py`, `perception.py`, `locations.json`, `navigation_graph.json`
Leads marks: Perception & navigation (3)

Task (definition of done)	Pts
Route planning over `navigation_graph.json` (BFS or Dijkstra) from the nearest node	2
Heading and arrival control with GPS and Compass; speed limit; documented arrival tolerance	3
Obstacle stop on ps0/1/6/7 → BLOCKED; no progress → TIMEOUT; stop the motors before any recovery	2
Detection: camera buffer → numpy → HSV mask → area, circularity, confidence (`detect()`)	3
Reject distractors: green ball, small red sphere, other shapes; document the thresholds	1
Bounded search policy: N views × K frames per location → FOUND / NOT_FOUND / UNCERTAIN	3

Provides to others
Navigator and search API (§2) · threshold and tolerance values with justification for the report

Consumes from others
Status constants (R3) · Supervisor for randomised trials (R1)

Boundary — does NOT: read Supervisor fields, DEF names or translations · teleport the robot · decide the next plan step · call the LLM

Own tests: `tests/test_perception.py`: detection on saved camera frames (red ball, green ball, distractor, empty view)

4 · Shared work

Item	Lead	Support
Evaluation runs across randomised resets (plan validity, search accuracy, distractor rejection, nav outcomes, times)	R2	all run trials
Report: own layer section (about 2 pages each)	all	—
Report: architecture overview (reuse diagram 1)	R3	—
Report: results & critical analysis	R2	R4 (search and nav data)
Report: limitations, ethics, privacy (speech, online models)	R1	—
Report: group contributions, references, final edit to 10 pages	R4	all proofread

Item	Lead	Support
Final ZIP check (no weights or keys, randomising Supervisor included)	R1	R3

5 · Working rules

- **Code review rotation:** R1 reviews R2 → R2 reviews R3 → R3 reviews R4 → R4 reviews R1. The reviewer must be able to explain the reviewed component in the demo.
- **Mocks first:** every role develops against mocks or fake data until integration. No one waits on another role to start.
- **Interface changes** (§2) are announced to the group **before** merging. R3 updates the validator and tests first.
- **Integration owner (R3)** keeps `project3_student.py` running at all times and merges sections from R1 and R2.
- **Open issue:** the starter package has **no setup Supervisor** (the red ball is fixed in the lab). R1 asks the teaching staff this week and uses the temporary Lab 2–based version until then.

6 · Timeline

Dates	Milestone
14–16 Oct	Interfaces (§2) frozen · Supervisor question sent · baseline validator tests passing · repo and branches set up
17–23 Oct	Each role builds its layer against mocks · own tests passing
24–27 Oct	Integration: one validated conditional path end-to-end, then one replanning path (blocking obstacle as trigger)
28–30 Oct	Full evaluation runs across randomised resets · report drafts · failure analysis
31 Oct – 1 Nov	Freeze demo configuration · final report edit · ZIP check · submit (one member uploads)